

Introduction to Systematic Design of Multi- Component Systems

SEAtS | 264726

Intended Learning Outcomes

By the end of this lecture, you will be able to:

- Identify the differences between monolithic and distributed design approaches.
- Describe the key principles of a systematic design process.
- Explain the importance of interfaces and coordination in distributed system development.
- Evaluate tools and methods used to support the systematic design of complex systems.
- Apply a systematic design process to a simplified real-world engineering problem.

Vocabulary: Four Related Terms

① System Theory

Knowledge about how systems work (general knowledge).

② Systems Thinking

Diagnosing a *specific* problem using that knowledge.

③ Systemic Design

Designing solutions that account for the whole system and its feedback loops.

④ Systematic Design

Breaking design into orderly, methodical steps — doesn't by itself require systems thinking.

Recap

Systemic Design

A design approach that integrates systems thinking into the design process to address complex, interconnected, socio-technical challenges involving multiple stakeholders and dynamic relationships.

Key characteristics:

- Emphasises relationships, feedback loops, and emergent behaviours
- Integrates qualitative and quantitative analysis
- Focused on sense-making and co-creation in messy, real-world contexts
- Often applied to sustainability, healthcare, policy, education, and ecosystems

Systemic Design Principles

A common systemic design mindset includes:

- Acknowledging interconnections
- Redefining waste as a resource
- Designing self-organising systems
- Using feedback as a guide to adaptive improvement

These mindsets are increasingly important as engineers face challenges that are not purely technical but socio-technical, requiring both human and machine systems to work harmoniously.

Systematic Design

Systematic Design is a structured approach that decomposes systems into interacting components, applying design principles to manage intricate challenges effectively.

1. **Structured:** Following repeatable processes with defined stages and checkpoints rather than intuitive approaches. This ensures consistency and quality across projects.
2. **Decomposable:** Breaking complex problems into manageable sub-problems that can be addressed independently, while maintaining awareness of interactions.
3. **Iterative:** Embracing feedback loops to continuously refine designs based on testing, simulation, and stakeholder input throughout the development lifecycle.

Systematic design is crucial for managing complexity, coordinating distributed teams, and ensuring that each subsystem contributes effectively to the global performance objectives. It is not simply a way to document a design process.

Key Characteristics

- Emphasises phased processes (e.g., requirements → architecture → implementation → testing)
- Often used in engineering and software development (e.g., V-model, waterfall models, MBSE)
- Supports modularisation, documentation, and project management
- Is about rigour and discipline in the design workflow

It stands in contrast to systemic design and systems thinking, which emphasise interconnections, feedback loops, and emergent behaviour in messy, socio-technical contexts rather than phased rigour, and to ad hoc design approaches, which rely on designer intuition and past experience without structured methodologies.

In multi-component systems, systematic design becomes essential because complexity grows exponentially with each additional component, and it draws on the practical toolkit captured in "Systematic Design Principles" and the monolithic vs distributed design choice for how the resulting process is coordinated.

Where systematic design is process-oriented and phased, systemic design is relationship-oriented and participatory, making it the bridge between systems thinking (the analytical lens) and the structured discipline of systematic design (the delivery process).

It is a foundational approach that ensures:

1. Reliability
2. Scalability
3. Integration

are established across subsystems when applied to multi-component systems. It integrates concepts from systems **engineering, software, and product design**.

Systematic design stands in contrast to ad hoc design approaches, which rely heavily on designer intuition and past experience without structured methodologies.

In multi-component systems, systematic design becomes essential due to the exponential growth in complexity with each additional component.

Key Principles of Systematic Design

Principle	Description
Structured Process Orientation	Follow a defined, repeatable sequence of design stages to ensure clarity and consistency.
Problem Decomposition	Break the system into manageable subsystems or components to reduce complexity.
Function-Driven Design	Begin with the desired functions and derive solutions that fulfill those functions.
Design Space Exploration and Evaluation	Generate multiple design alternatives and evaluate them using clear, objective criteria.
Interface Definition and Integration Planning	Clearly define interfaces between components to ensure compatibility and successful integration.
Requirements Traceability	Link every design decision back to a functional or non-functional requirement.
Iterative Refinement and Feedback Loops	Use test results and stakeholder feedback to iteratively improve the design.
Documentation and Knowledge Management	Maintain thorough records of decisions, risks, trade-offs, and changes throughout the process.
Design for Lifecycle Considerations	Consider manufacturability, maintainability, and sustainability from the start.
Use of Formal Tools and Models	Apply CAD, SysML, simulation, and modelling tools to support rigour, precision, and communication.

Social Media Algorithm Example

A social media platform contains three coupled components:

1. **The recommendation algorithm** — learns from user engagement, ranks content
2. **User behaviour** — users see recommendations and decide what to engage with
3. **Content ecosystem** — posts, creators, discourse shaped by what algorithms amplify

These are not independent. They form a feedback loop.

How It Works

Initial state: Algorithm recommends diverse content based on past engagement.

Positive feedback loop:

- User engages with political content (likes, shares, comments)
- Algorithm learns: this user likes political content
- Algorithm recommends more political content to this user
- More political engagement trains algorithm further
- Algorithm recommends even more political content

Emergence: After weeks, the user's feed contains nearly only political content—often increasingly extreme versions. No engineer programmed polarization. No single decision caused it. It emerged from the feedback loop between algorithm and user behaviour.

System-level effect: Across millions of users, this creates filter bubbles, where people see only views that confirm their existing beliefs. Discourse fragments. Misinformation spreads in bubbles. Trust in institutions erodes.

Why Traditional Thinking Fails

Optimizing the algorithm alone won't fix this:

- Tweaking recommendations to "show more diverse content" is ignored if users ignore it (user behaviour is part of the system)
- Removing engagement signals changes incentives but creates new feedback loops elsewhere
- Individual features work fine in isolation; system behaviour is unpredictable

The System Theory Insight

You cannot understand the algorithm's impact without understanding the user–algorithm feedback loop as a single system. The algorithm and user behaviour are interdependent. Change one part and you destabilize the whole. Emergent behaviours (polarization, filter bubbles) come from the system, not from any component.

Solutions require systems thinking:

- Add negative feedback (e.g., expose users to opposing views, but measure actual engagement changes)
- Change incentive structures across the entire platform, not just the algorithm
- Monitor emergent behaviours, not just algorithm metrics

Worked Example: A Systemic Redesign Process

Where systems-thinking *diagnoses* the algorithm/user feedback loop that produces filter bubbles , systemic design *redesigns the whole system*, not just the algorithm:

- Change recommendation weights to penalise extreme content
- Redesign the UI to surface opposing viewpoints and trigger reflection
- Change creator incentives from engagement-maximisation to quality
- Add circuit breakers that inject diverse content when a feed becomes too homogeneous
- Monitor for emergent polarisation across the whole platform

These interventions only work *together*: changing the algorithm without changing incentives still lets creators game the system; showing opposing views without changing the algorithm gets them ignored. Systemic design means designing across the system, not one component at a time — systematic-design for how the same scenario looks run as a phased process instead.

Worked Example: A Systematic Redesign Process

Continuing the same scenario as systemic-design's worked example, a systematic approach to redesigning a recommender system looks like an ordered checklist:

1. Audit the current recommendation algorithm
2. Design a new weighting scheme
3. Build A/B test infrastructure
4. Run tests on 10% of users
5. Measure engagement metrics
6. Roll out to 100% if metrics improve

Each step is methodical and auditable — but, taken alone, isolated: step 1 doesn't show how it affects step 4, or how improving one metric breaks another part of the system. That's the gap systemic-design closes.

Recap: Systemic vs. Systematic Design

Recall from the Systems Thinking lecture: **systemic design** accounts for the whole system and its feedback loops; **systematic design** breaks it into orderly, methodical steps. This lecture goes deep on the second.

Systematic Design Process Overview

The systematic design process can be represented in the following stages:

1. Inquiry – Understand system context, user needs, and constraints
2. Framing & Requirements Gathering – Define the scope, boundaries, and stakeholder expectations
3. System Architecture & Interface Definition – Decompose system into components; define interaction contracts
4. Component Design & Optimisation – Apply appropriate methods (e.g., FEA, simulation, surrogate modelling)
5. Integration & Testing – Assemble components, verify performance against system requirements
6. Feedback & Iteration – Use simulation and real-world data to refine models

Tools Supporting Systematic Design

To support these phases, engineers and designers use a range of tools:

1) Prediction / surrogate models

2) Optimization Algorithms

3) Simulation and Validation

4) System Mapping Tools

5) Prototyping and Implementation

Understanding Multi-Component Systems

A multi-component system is composed of physically or functionally distinct elements that work together to deliver a complete functionality. Effective system design requires balancing these elements:

1. System-level requirements that address the overall purpose
2. Component-level specifications that drive individual elements
3. Interface definitions that ensure compatibility

Design Drivers

1. Performance metrics (speed, accuracy, efficiency)
2. Cost constraints (materials, manufacturing, lifecycle)
3. Modularity requirements (maintenance, upgradeability)
4. Time-to-market pressures

Challenges in Systematic Design of Multi-Component Systems

Challenge	Impact
Interface complexity	Integration failures, rework
Cross-disciplinary coordination	Misalignment, communication breakdown
Requirements instability	Design churn, rework
Scalability	Process overhead and design delays
Limited design exploration	Missed innovation opportunities
Tool integration	Inefficiencies, data loss
Change propagation	System-wide knock-on effects
Testing bottlenecks	Undetected errors, delayed feedback
Traceability management	High documentation cost
Human factors	Non-compliance, lack of buy-in

Interface complexity

A key challenge in multi-component systems is managing the interfaces between components. Interfaces define how data, energy, or force is exchanged between components and are critical to system functionality.

Mechanical Design

In mechanical design, interface stiffness matrices describe how forces and displacements are related across component boundaries.

Software Systems

In software systems, APIs define function calls and data structures.

Embedded Systems

In embedded systems, protocols govern timing and synchronization.

This allows each component to be optimized independently, However, such decomposition risks missing globally optimal solutions or producing physically infeasible designs.

Monolithic vs Distributed Design

Monolithic design

Monolithic design approaches attempt to optimize all design variables simultaneously in a single, tightly integrated process. This has the advantage of potentially achieving globally optimal solutions but suffers from scalability issues and coordination challenges in practice.

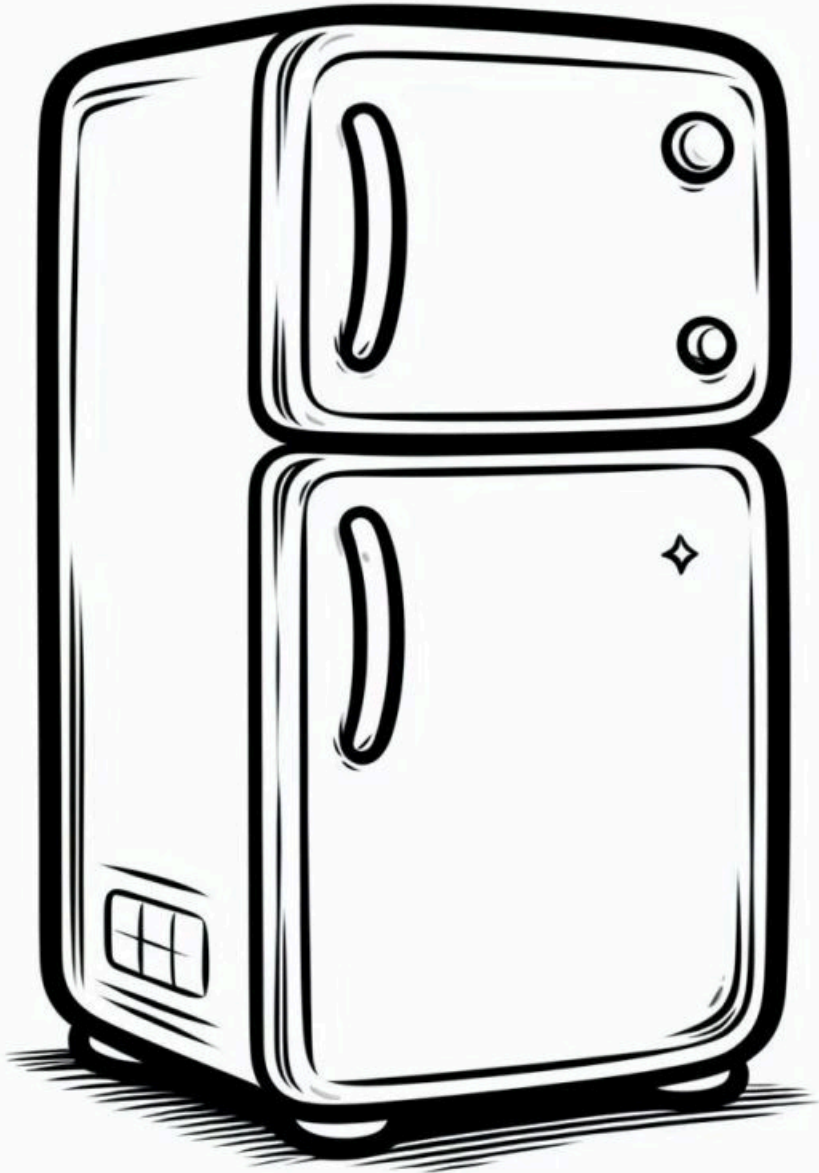
Distributed design

Distributed design optimization, on the other hand, decomposes the global design problem into subproblems, each associated with a component or subsystem. Coordination is then required to ensure that the subproblem solutions are compatible and converge to a system-level optimum.

This shift from global to local design parallels organisational design: individual teams can work on parts as long as shared protocols and interfaces are respected.

Case Study

Is it about keep things cool?



Bullet Proof Fridges

i 1920s, 30's

- First household electric fridges became commercially available, replacing iceboxes.
- Introduced sealed refrigerant systems (using toxic gases at first, later safer ones).
- Improved food safety by slowing spoilage.

i 1940s

- Mass adoption after WWII, as manufacturing shifted back to consumer goods.
- More reliable compressors and safer refrigerants (Freon).
- First freezer compartments introduced for limited frozen storage.

i 1950s

- Fridges became a staple in middle-class homes.
- Chrome finishes, rounded designs, and interior lighting added.
- Larger freezer sections allowed frozen food markets to grow.

built with heavy steel, thick insulation, and very long lifespans (some lasted 40+ years).

The Cheap Fridges

i 1960s

- Automatic defrosting (no more manual ice scraping).
- Adjustable shelves and door storage.
- More efficient insulation and compressors reduced energy use compared to earlier models.

i 1970s

- Energy crisis led to new efficiency standards.
- Introduction of tighter door seals and improved insulation.
- More focus on durability and safety of refrigerants.

i 1980s

- Side-by-side fridge/freezer designs became popular.
- Ice and water dispensers built into the door.
- Energy-efficient models introduced due to stricter regulations.

Slightly lower quality but designs shifted toward convenience and style (ice makers, side-by-side doors), which sometimes reduced durability.

The Compliance Fridges

① 1990s

- Microprocessor-based controls for precise temperature regulation.
- Use of CFC-free refrigerants (environmental shift after Montreal Protocol).
- Better humidity control for fresh produce.

① 2000s

- Sleeker designs with stainless steel finishes.
- Multi-zone cooling systems (different temperatures for different compartments).
- Introduction of energy ratings and eco-labels guiding consumer choice.

① 2010s

- Wi-Fi-enabled “smart fridges” with touchscreens, cameras, and connectivity to smartphones.
- Inverter compressors for quieter, more efficient cooling.
- LED lighting and advanced insulation further cut energy use.

more electronics and plastic parts made repairs harder and sometimes reduced lifespan compared to earlier “bulletproof” models.



Conclusion

Systematic design provides a structured and methodical way to manage the complexity inherent in large, multi-component systems.

It is both a mindset and a methodology, drawing from systems thinking, optimisation, and domain-specific engineering practices.

By integrating stakeholder needs, technical constraints, and adaptive feedback, systematic design becomes essential not just for building systems that work, but for building systems that can evolve.

Thank you

Dr Nawfal Al Hashimy

Lecturer in Cyber Physical Security

University of Southampton | Faculty of Engineering and Physical
Sciences

Cyber Physical Systems Research Group

☎ +44 (0) 23 8059 6659

@ Nawfal@soton.ac.uk

Highfield Campus, Building 53, Room 4049
Burgess Road, Southampton SO17 1BJ, UK

